Rishabh Software Pvt. Ltd., Vadodara

Technical Design Document of Pipeline

Title of Assessment:

“Real Estate Analytics Data Warehouse”

Submitted By:

Vipul Patel

BI & Data Engineering Intern, Rishabh Software

Intern ID: 1115

Submitted To:

Rishabh Software Pvt. Ltd., Vadodara

Under the Guidance of:

Mr. Jaiminkumar Rabari

Rishabh Software Pvt. Ltd.

Contents

1. Project Overview

Project Title: Real Estate Analytics Data Warehouse — Azure Data Factory Pipeline
Prepared By: Vipul Patel
Submission Date: 21 May 2026

1.1 Objective

The objective of this project is to design and implement a fully automated data warehouse solution for a real estate organization using Microsoft Azure cloud services. The solution ingests raw data from multiple source systems, transforms and cleanses it, and loads it into a structured star schema data warehouse that enables the analytics team to build dashboards and generate business insights.

1.2 Business Problem

The real estate organization manages data across multiple systems including agent records, property listings, client leads, campaign marketing, property viewings, offers, and transactions. This data was stored in separate CSV files with no centralized reporting layer. The analytics team had no clean and structured dataset to build reports on. Key business questions that could not be answered included:

- Which agents are generating the highest revenue?
- Which properties are selling fastest?
- Which marketing campaigns are converting the most leads?
- What is the average time to close a transaction by region?
- How many leads convert to viewings and then to actual sales?

This project solves all of these problems by building a centralized, clean, and analytics-ready data warehouse.

1.3 Scope

In Scope:

- Ingestion of 8 CSV source files into Azure Blob Storage
- Loading raw data into SQL Staging tables via ADF Copy Activity
- Transforming and loading clean data into Star Schema DWH tables via Stored Procedures
- Implementation of SCD Type 1 and SCD Type 2 for dimension tables
- Creation of 4 analytical views for the Power BI team
- Pipeline logging and monitoring
- Data quality validation
- GitHub version control integration

1.4 Source Systems and Data

The following 8 CSV files were provided as source data:

File Name	Description	Row Count
Agents.csv	Real estate agent details	500
Properties.csv	Property master data	8,000
Listings.csv	Property listing records	12,060
Campaigns.csv	Marketing campaign details	1,500

Leads.csv	Client lead records	50,000
Viewings.csv	Property viewing records	40,000
Offers.csv	Offer records per listing	25,000
Transactions.csv	Completed sale transactions	8,472

Total Source Records: 145,532

1.5 Technologies Used

Technology	Purpose
Azure Resource Group	Container for all project resources
Azure Blob Storage	Raw CSV file storage
Azure SQL Database	Staging and Data Warehouse database
Azure Data Factory	Pipeline orchestration and data movement
Azure Key Vault	Secure credential storage
SQL Server Stored Procedures	Data transformation and loading
GitHub	Version control for ADF pipelines
SQL Views	Analytical layer for Power BI team

1.6 Project Outcome

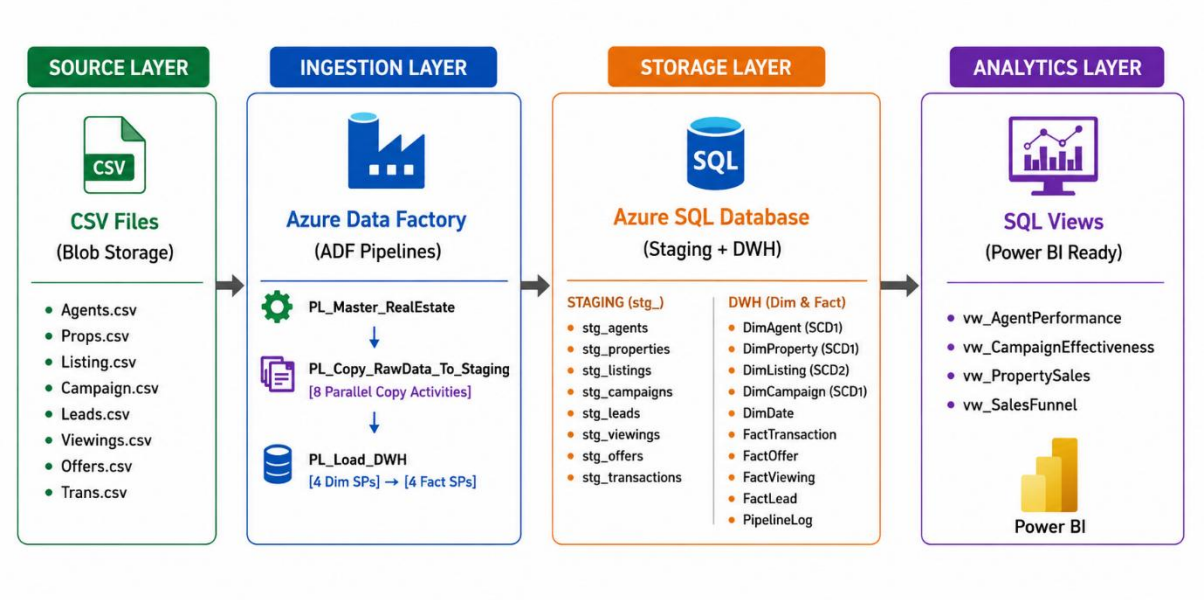
At the end of this project the following deliverables are available:

- A fully operational Azure cloud data pipeline
- 8 clean staging tables with raw source data
- 5 dimension tables and 4 fact tables in star schema format
- SCD Type 1 implemented on DimAgent, DimProperty, DimCampaign
- SCD Type 2 implemented on DimListing with full history tracking
- 3 ADF pipelines organized in folders with GitHub version control
- 4 analytical SQL views ready for Power BI consumption
- Complete pipeline logging and monitoring
- Full data quality validation with zero errors

2. Architecture Overview

2.1 High Level Architecture

The solution follows a classic three layer data warehouse architecture built entirely on Microsoft Azure cloud platform. Data flows from source CSV files through a staging layer into a final data warehouse layer where it is made available to the analytics team.



2.2 Detailed Data Flow

Stage 1 — Source to Blob Storage All 8 CSV files are uploaded manually to Azure Blob Storage in a container named raw-data. This is a one time upload for initial load. For subsequent loads only new or updated CSV files need to be uploaded.

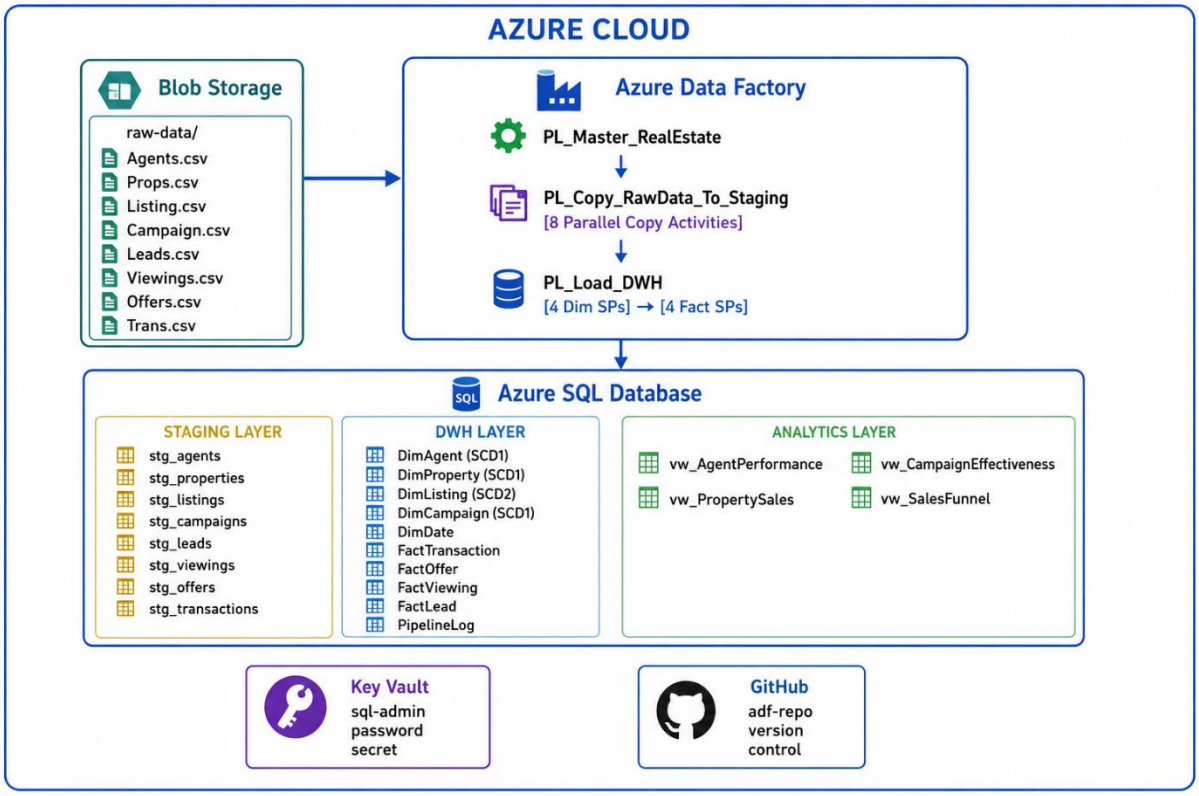
Stage 2 — Blob Storage to Staging (Copy Activity) ADF Pipeline PL_Copy_RawData_To_Staging reads each CSV file from Blob Storage using 8 parallel Copy Activities and loads raw data into 8 SQL Staging tables. Each staging table is truncated before loading to ensure fresh data on every run. All 8 Copy Activities run in parallel for maximum performance.

Stage 3 — Staging to DWH (Stored Procedures) ADF Pipeline PL_Load_DWH executes 8 Stored Procedures via Stored Procedure Activities. The 4 Dimension SPs run in parallel first. Once all Dimensions complete successfully, the 4 Fact SPs run in parallel. This order ensures surrogate keys exist in Dimension tables before Fact tables reference them.

Stage 4 — DWH to Analytics Layer (SQL Views) 4 SQL Views are created on top of DWH tables. These views pre-join Dimension and Fact tables and present clean, analytics-ready datasets. The Power BI team connects directly to these views without writing any SQL.

Stage 5 — Master Pipeline Orchestration ADF Pipeline PL_Master_RealEstate orchestrates both pipelines in correct sequence. It first triggers PL_Copy_RawData_To_Staging and waits for completion before triggering PL_Load_DWH. This ensures data integrity at every stage.

2.3 Architecture Diagram



2.4 Why This Architecture

Blob Storage is the most cost effective and scalable storage solution on Azure. It supports any file format including CSV and integrates natively with ADF Copy Activity. LRS redundancy was chosen to minimize cost while maintaining data availability.

Staging layer acts as a safe landing zone for raw data. If any transformation fails the raw data is preserved in staging and pipeline can be re-run without re-uploading CSV files. DWH layer contains clean, transformed, and structured data ready for analytics.

Source data was largely clean with only minor issues such as NULL values and data type mismatches. These were handled efficiently inside Stored Procedures using CAST and ISNULL functions. Stored Procedures are faster, cheaper, and easier to maintain than ADF Mapping Data Flows for this use case.

Star schema is the industry standard for data warehouse design. It provides fast query performance, simple joins, and easy understanding for analytics teams. Dimension tables store descriptive attributes and Fact tables store measurable events linked via surrogate keys.

Hardcoding passwords in ADF Linked Services is a security risk. Key Vault stores the SQL password as a secret and ADF retrieves it at runtime using Managed Identity. This follows enterprise security best practices.

GitHub provides version control for all ADF pipeline JSON definitions. Any changes to pipelines are tracked, can be reviewed, and can be rolled back if needed. This follows DevOps best practices for data engineering projects.

3. Azure Resources

3.1 Resource Group

Field	Value
Name	rg-realestate-analytics
Type	Resource Group
Region	Central India
Purpose	Container for all project resources

3.2 Storage Account

Field	Value
Name	strealestateanalytics
Type	Azure Blob Storage
Region	Central India
Performance	Standard
Redundancy	Locally Redundant Storage (LRS)
Container Name	raw-data
Purpose	Store all 8 source CSV files

Files Uploaded:

- Agents.csv
- Properties.csv
- Listings.csv
- Campaigns.csv
- Leads.csv
- Viewings.csv
- Offers.csv
- Transactions.csv

3.3 Azure SQL Server and Database

SQL Server:

Field	Value
Name	sql-realestate-server
Type	Azure SQL Server
Region	Central India
Authentication	SQL Authentication
Admin Login	sqladmin
Purpose	Host the SQL Database

SQL Database:

Field	Value
Name	db-realestate-dw
Type	Azure SQL Server
Pricing Tier	Basic
DTUs	5

Storage	2GB
Redundancy	Locally Redundant
Purpose	Staging tables + DWH tables + Views

3.4 Azure Key Vault

Field	Value
Name	kv-realestate-prod
Type	Azure Key Vault
Region	Central India
Pricing Tier	Standard
Purpose	Secure storage of SQL admin password

Secret Stored:

Secret Name	Value
sql-admin-password	SQL Server admin password

Access Configuration:

- Role Assignment: Key Vault Administrator assigned to project owner
- Role Assignment: Key Vault Secrets User assigned to ADF Managed Identity

3.5 Azure Data Factory

Field	Value
Name	adf-realestate-prod
Type	Azure Data Factory V2
Region	Central India
Purpose	Pipeline orchestration, data movement
GitHub Repo	adf-realestate-analytics
Collaboration Branch	main
Publish Branch	adf_publish

ADF Components Created:

Component	Count	Details
Linked Services	3	LS_KeyVault, LS_BlobStorage, LS_AzureSQLDB
Datasets	16	8 Blob Source + 8 SQL Staging Sink
Pipelines	3	PL_Copy_RawData_To_Staging, PL_Load_DWH, PL_Master_RealEstate

Pipeline Folders:

Folder	Pipeline
01_Staging	PL_Copy_RawData_To_Staging
02_DWH_Load	PL_Load_DWH
03_Master	PL_Master_RealEstate

Dataset Folders:

Folder	Datasets
Blob_Source	DS_Blob_Agents, DS_Blob_Properties, DS_Blob_Listings, DS_Blob_Campaigns, DS_Blob_Leads, DS_Blob_Viewings, DS_Blob_Offers, DS_Blob_Transactions
SQL_Staging	DS_SQL_stg_agents, DS_SQL_stg_properties, DS_SQL_stg_listings, DS_SQL_stg_campaigns, DS_SQL_stg_leads, DS_SQL_stg_viewings, DS_SQL_stg_offers, DS_SQL_stg_transactions

4. Database Design

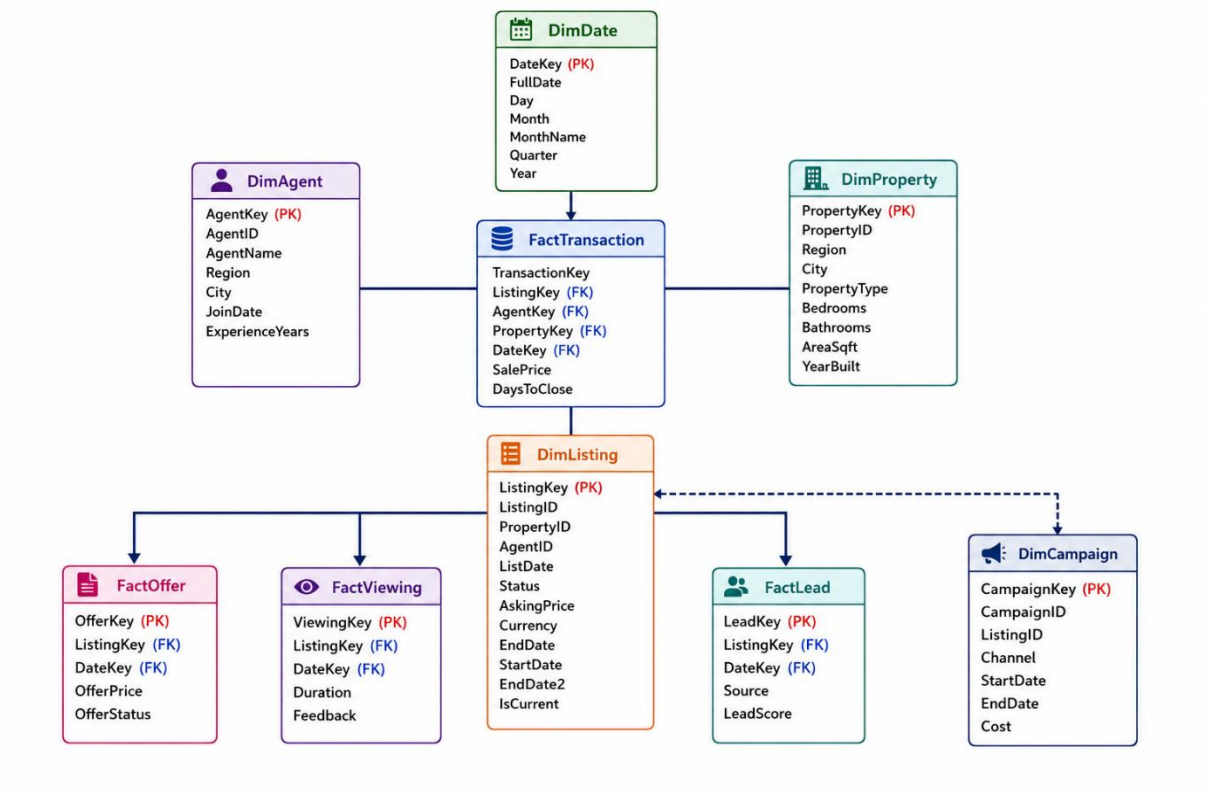
4.1 Star Schema Overview

The data warehouse follows a Star Schema design which is the industry standard for analytical databases. In a star schema one or more Fact tables sit at the center and are surrounded by Dimension tables. Fact tables store measurable business events and Dimension tables store descriptive attributes about those events.

Why Star Schema?

- Simple and intuitive structure for analytics teams
- Fast query performance due to minimal joins
- Easy to understand for Power BI report builders
- Industry standard recognized by all data professionals
- Supports both simple and complex analytical queries

4.2 Star Schema Diagram



4.3 Staging Tables

Staging tables are raw landing tables that mirror the structure of source CSV files exactly. No transformations are applied at this stage. All date columns are stored as VARCHAR to accept any date format from source. Staging tables are truncated before each pipeline run.

stg_agents

Column	Data Type
AgentID	VARCHAR(10)
AgentName	VARCHAR(100)
Region	VARCHAR(50)
City	VARCHAR(50)
JoinDate	VARCHAR(20)
ExperienceYears	INT

stg_properties

Column	Data Type
PropertyID	VARCHAR(10)
Region	VARCHAR(50)
City	VARCHAR(50)
PropertyType	VARCHAR(50)
Bedrooms	INT
Bathrooms	INT
AreaSqft	INT
YearBuilt	INT

stg_listings

Column	Data Type
ListingID	VARCHAR(10)
PropertyID	VARCHAR(10)
AgentID	VARCHAR(10)
ListDate	VARCHAR(20)
Status	VARCHAR(20)
AskingPrice	DECIMAL(18,2)
Currency	VARCHAR(5)
EndDate	VARCHAR(20)

stg_campaigns

Column	Data Type
CampaignID	VARCHAR(10)
ListingID	VARCHAR(10)
StartDate	VARCHAR(20)
EndDate	VARCHAR(20)
Channel	VARCHAR(50)
Cost	DECIMAL(18,2)

stg_leads

Column	Data Type
LeadID	VARCHAR(10)
ListingID	VARCHAR(10)
LeadDate	VARCHAR(20)
Source	VARCHAR(50)
LeadScore	DECIMAL(5,2)

stg_viewings

Column	Data Type
ViewingID	VARCHAR(10)
ListingID	VARCHAR(10)
ViewingDate	VARCHAR(20)
DurationMinutes	INT
Feedback	VARCHAR(20)

stg_offers

Column	Data Type
OfferID	VARCHAR(10)
ListingID	VARCHAR(10)
OfferDate	VARCHAR(20)
OfferPrice	DECIMAL(18,2)
OfferStatus	VARCHAR(20)

stg_transactions

Column	Data Type
TransactionID	VARCHAR(10)
ListingID	VARCHAR(10)
AgentID	VARCHAR(10)
PropertyID	VARCHAR(10)
OfferDate	VARCHAR(20)
ClosingDate	VARCHAR(20)
SalePrice	DECIMAL(18,2)

4.4 DWH Dimension Tables

Dimension tables contain clean, transformed, and deduplicated data with surrogate keys assigned via IDENTITY(1,1). Natural keys from source are preserved alongside surrogate keys for traceability.

DimAgent — SCD Type 1

Column	Data Type	Constraint
AgentKey	INT	PK, IDENTITY(1,1)
AgentID	VARCHAR(10)	
AgentName	VARCHAR(100)	
Region	VARCHAR(50)	
City	VARCHAR(50)	
JoinDate	DATE	

ExperienceYears	INT	

DimProperty — SCD Type 1

Column	Data Type	Constraint
PropertyKey	INT	PK, IDENTITY(1,1)
PropertyID	VARCHAR(10)	
Region	VARCHAR(50)	
City	VARCHAR(50)	
PropertyType	VARCHAR(50)	
Bedrooms	INT	
Bathrooms	INT	
AreaSqft	INT	
YearBuilt	INT	

DimListing — SCD Type 2

Column	Data Type	Constraint
ListingKey	INT	PK, IDENTITY(1,1)
ListingID	VARCHAR(10)	
PropertyID	VARCHAR(10)	
AgentID	VARCHAR(10)	
ListDate	DATE	
Status	VARCHAR(20)	
AskingPrice	DECIMAL(18,2)	
Currency	VARCHAR(5)	
EndDate	DATE	
StartDate	DATE	
EndDate2	DATE	
IsCurrent	BIT	

DimCampaign — SCD Type 1

Column	Data Type	Constraint
CampaignKey	INT	PK, IDENTITY(1,1)
CampaignID	VARCHAR(10)	
ListingID	VARCHAR(10)	
Channel	VARCHAR(10)	
StartDate	DATE	
EndDate	VARCHAR(20)	
Cost	DECIMAL(18,2)	

DimDate — Static Reference Table

Column	Data Type	Constraint
DateKey	INT	PK
FullDate	DATE	
Day	INT	

Month	INT	
MonthName	VARCHAR(20)	
Quarter	INT	
Year	INT	

Date range: 2023-01-01 to 2026-12-31 — Total 1,461 rows generated via SQL loop.

4.5 DWH Fact Tables

Fact tables store measurable business events. They reference Dimension tables via surrogate keys as foreign keys. Fact tables use incremental load with duplicate protection.

FactTransaction — Core Sales Fact

Column	Data Type	Constraint
TransactionKey	INT	PK, IDENTITY(1,1)
TransactionID	VARCHAR(10)	
ListingKey	INT	FK → DimListing
AgentKey	INT	FK → DimAgent
PropertyKey	INT	FK → DimProperty
DateKey	INT	FK → DimDate
SalePrice	DECIMAL(18,2)	
DaysToClose	INT	

FactOffer

Column	Data Type	Constraint
OfferKey	INT	PK, IDENTITY(1,1)
OfferID	VARCHAR(10)	
ListingKey	INT	FK → DimListing
DateKey	INT	FK → DimDate
OfferPrice	DECIMAL(18,2)	
OfferStatus	VARCHAR(20)	

FactViewing

Column	Data Type	Constraint
ViewingKey	INT	PK, IDENTITY(1,1)
ViewingID	VARCHAR(10)	
ListingKey	INT	FK → DimListing
DateKey	INT	FK → DimDate
DurationMinutes	INT	
Feedback	VARCHAR(20)	

FactLead

Column	Data Type	Constraint
LeadKey	INT	PK, IDENTITY(1,1)
LeadID	VARCHAR(10)	
ListingKey	INT	FK → DimListing
DateKey	INT	FK → DimDate
Source	VARCHAR(50)	
LeadScore	DECIMAL(5,2)	

4.6 PipelineLog Table

Column	Data Type
LogID	INT IDENTITY(1,1)
PipelineName	VARCHAR(100)
RunStatus	VARCHAR(20)
RowsCopied	INT
StartTime	DATETIME
EndTime	DATETIME
ErrorMessage	VARCHAR(500)

4.7 Database Indexes

Indexes were created on all foreign key columns in Fact tables to improve join performance for analytical queries.

Index Name	Table	Column
IX_FactTransaction_ListingKey	FactTransaction	ListingKey
IX_FactTransaction_AgentKey	FactTransaction	AgentKey
IX_FactTransaction_PropertyKey	FactTransaction	PropertyKey
IX_FactOffer_ListingKey	FactOffer	ListingKey
IX_FactViewing_ListingKey	FactViewing	ListingKey
IX_FactLead_ListingKey	FactLead	ListingKey
IX_DimListing_ListingID	DimListing	ListingID

5. Linked Services and Datasets

5.1 Linked Services Overview

Linked Services in Azure Data Factory define the connection information to external data sources and destinations. They act like connection strings that tell ADF where to connect and how to authenticate. Three Linked Services were created for this project.

5.2 LS_KeyVault

Field	Value
Name	LS_KeyVault
Type	Azure Key Vault
Key Vault Name	kv-realestate-prod
Authentication	Managed Identity
Purpose	Retrieve SQL password secret at runtime

ADF uses its system assigned Managed Identity to authenticate to Key Vault without any username or password. Key Vault Secrets User role was assigned to ADF Managed Identity via IAM Role Assignment.

5.3 LS_BlobStorage

Field	Value
Name	LS_BlobStorage
Type	Azure Blob Storage

Storage Account	strealestateanalytics
Authentication	Account Key
Purpose	Read CSV source files from Blob container

ADF connects to the storage account using the account key and reads files from the raw-data container. All 8 Blob Source datasets reference this Linked Service.

5.4 LS_AzureSQLDB

Field	Value
Name	LS_AzureSQLDB
Type	Azure SQL Database
Server	sql-realestate-server
Database	db-realestate-dw
Authentication	SQL Authentication
Username	sqladmin
Password	Retrieved from LS_KeyVault → secret: sql-admin-password
Purpose	Read and write data to SQL staging and DWH tables

Instead of hardcoding the SQL password in the Linked Service configuration, the password is stored as a secret in Key Vault and referenced at runtime. This means the password is never visible in ADF Studio or in GitHub repository — a mandatory security practice in enterprise environments.

5.5 Datasets Overview

Datasets in ADF define the structure and location of data within a Linked Service. 16 datasets were created and organized in 2 folders.

5.6 Blob Source Datasets — Folder: Blob_Source

All 8 Blob datasets share the same configuration with only file name changing per dataset.

Common Configuration:

Field	Value
Type	Azure Blob Storage — DelimitedText
Linked Service	LS_BlobStorage
Container	raw-data
First Row as Header	Yes
Import Schema	From connection/store

Dataset List:

Dataset Name	File Name	Row Count
DS_Blob_Agents	Agents.csv	500
DS_Blob_Properties	Properties.csv	8,000
DS_Blob_Listings	Listings.csv	12,060
DS_Blob_Campaigns	Campaigns.csv	1,500
DS_Blob_Leads	Leads.csv	50,000
DS_Blob_Viewings	Viewings.csv	40,000
DS_Blob_Offers	Offers.csv	25,000
DS_Blob_Transactions	Transactions.csv	8,472

5.7 SQL Staging Sink Datasets — Folder: SQL_Staging

All 8 SQL datasets share the same configuration with only table name changing per dataset.

Common Configuration:

Field	Value
Type	Azure SQL Database
Linked Service	LS_AzureSQLDB
Import Schema	From connection/store

Dataset List:

Dataset Name	File Name
DS_SQL_stg_agents	stg_agents
DS_SQL_stg_properties	stg_properties
DS_SQL_stg_listings	stg_listings
DS_SQL_stg_campaigns	stg_campaigns
DS_SQL_stg_leads	stg_leads
DS_SQL_stg_viewings	stg_viewings
DS_SQL_stg_offers	stg_offers
DS_SQL_stg_transactions	stg_transactions

6. Stored Procedures

6.1 Overview

Nine Stored Procedures were created in Azure SQL Database to handle all data transformation and loading operations. Eight procedures load data from Staging tables into DWH tables and one procedure handles pipeline logging.

6.2 SCD Type 1 — Explanation

SCD stands for Slowly Changing Dimension. Type 1 means when a value changes in source data the old value in DWH is overwritten with the new value. No history is kept.

Applied to**:** DimAgent, DimProperty, DimCampaign

Why Type 1 for these tables?

- Agent city or region change — analytics team needs current location only
- Property details change — current details are what matter for reporting
- Campaign cost update — latest cost is the correct value for ROI calculation

Logic:

1. UPDATE existing records where any column value has changed
2. INSERT new records that do not exist yet

6.3 SCD Type 2 — Explanation

SCD Type 2 means when a value changes in source data a new record is inserted with the new values and the old record is marked as expired. Full history is preserved.

Applied to: DimListing

Why Type 2 for DimListing? Listing status changes such as Active → Under Offer → Sold represent important business events. Analytics team needs to track the full lifecycle of a listing over time. Losing this history would mean losing visibility into how long a property stayed in each status.

Additional Columns Added for SCD Type 2:

Column	Purpose
StartDate	Date this version of the record became active
EndDate2	Date this version expired — NULL if still current
IsCurrent	1 = current record, 0 = historical record

Logic:

1. EXPIRE old record — set IsCurrent = 0 and EndDate2 = today where values changed
2. INSERT new record — with new values, IsCurrent = 1, EndDate2 = NULL

6.4 sp_Load_DimAgent

Type: SCD Type 1 **Source:** stg_agents **Target:** DimAgent

```
CREATE PROCEDURE sp_Load_DimAgent
AS
BEGIN
  -- Update changed records (SCD Type 1)
  UPDATE d
  SET
    d.AgentName = ISNULL(s.AgentName, d.AgentName),
    d.Region = ISNULL(s.Region, 'Unknown'),
    d.City = ISNULL(s.City, 'Unknown'),
    d.JoinDate = CAST(s.JoinDate AS DATE),
    d.ExperienceYears = ISNULL(s.ExperienceYears, 0)
  FROM DimAgent d
  INNER JOIN stg_agents s ON d.AgentID = s.AgentID
  WHERE
    d.Region != ISNULL(s.Region, 'Unknown') OR
    d.City != ISNULL(s.City, 'Unknown') OR
    d.ExperienceYears != ISNULL(s.ExperienceYears, 0);

  -- Insert new records only
  INSERT INTO DimAgent
    (AgentID, AgentName, Region, City, JoinDate, ExperienceYears)
  SELECT DISTINCT
    AgentID,
```

```
    AgentName,
    ISNULL(Region, 'Unknown'),
    ISNULL(City, 'Unknown'),
    CAST(JoinDate AS DATE),
    ISNULL(ExperienceYears, 0)
  FROM stg_agents
  WHERE AgentID NOT IN (SELECT AgentID FROM DimAgent);
END
```

6.5 sp_Load_DimProperty

Type: SCD Type 1 **Source:** stg_properties **Target:** DimProperty

```
CREATE PROCEDURE sp_Load_DimProperty
AS
BEGIN
  -- Update changed records (SCD Type 1)
  UPDATE d
  SET
    d.Region = ISNULL(s.Region, 'Unknown'),
    d.City = ISNULL(s.City, 'Unknown'),
    d.PropertyType = ISNULL(s.PropertyType, 'Unknown'),
    d.Bedrooms = ISNULL(CAST(s.Bedrooms AS INT), 0),
    d.Bathrooms = ISNULL(CAST(s.Bathrooms AS INT), 0),
    d.AreaSqft = ISNULL(s.AreaSqft, 0),
    d.YearBuilt = ISNULL(s.YearBuilt, 0)
  FROM DimProperty d
  INNER JOIN stg_properties s ON d.PropertyID = s.PropertyID
  WHERE
    d.Region != ISNULL(s.Region, 'Unknown') OR
    d.City != ISNULL(s.City, 'Unknown') OR
    d.PropertyType != ISNULL(s.PropertyType, 'Unknown') OR
    d.Bedrooms != ISNULL(CAST(s.Bedrooms AS INT), 0) OR
    d.Bathrooms != ISNULL(CAST(s.Bathrooms AS INT), 0);

  -- Insert new records only
  INSERT INTO DimProperty
    (PropertyID, Region, City, PropertyType,
     Bedrooms, Bathrooms, AreaSqft, YearBuilt)
  SELECT DISTINCT
    PropertyID,
    ISNULL(Region, 'Unknown'),
    ISNULL(City, 'Unknown'),
    ISNULL(PropertyType, 'Unknown'),
    ISNULL(CAST(Bedrooms AS INT), 0),
```

```
    ISNULL(CAST(Bathrooms AS INT), 0),
    AreaSqft,
    YearBuilt
  FROM stg_properties
  WHERE PropertyID NOT IN (SELECT PropertyID FROM DimProperty);
END
```

6.6 sp_Load_DimListing

Type: SCD Type 2 **Source:** stg_listings **Target:** DimListing

```
CREATE PROCEDURE sp_Load_DimListing
AS
BEGIN
  -- Step A: Expire old records where values changed
  UPDATE DimListing
  SET
    IsCurrent = 0,
    EndDate2 = GETDATE()
  WHERE IsCurrent = 1
  AND ListingID IN (
    SELECT s.ListingID
    FROM stg_listings s
    INNER JOIN DimListing d ON s.ListingID = d.ListingID
    WHERE d.IsCurrent = 1
    AND (
      d.Status != s.Status OR
      d.AskingPrice != s.AskingPrice OR
      d.Currency != s.Currency OR
      ISNULL(CAST(d.EndDate AS VARCHAR), '') !=
      ISNULL(s.EndDate, '')
    )
  );

  -- Step B: Insert new and changed records
  INSERT INTO DimListing
    (ListingID, PropertyID, AgentID, ListDate, Status,
     AskingPrice, Currency, EndDate, StartDate, EndDate2, IsCurrent)
  SELECT DISTINCT
    s.ListingID,
    ISNULL(s.PropertyID, 'Unknown'),
    ISNULL(s.AgentID, 'Unknown'),
    CAST(s.ListDate AS DATE),
    ISNULL(s.Status, 'Unknown'),
    ISNULL(s.AskingPrice, 0),
```

```
    ISNULL(s.Currency, 'Unknown'),
    CASE
      WHEN s.EndDate IS NULL OR s.EndDate = '' THEN NULL
      ELSE CAST(s.EndDate AS DATE)
    END,
    GETDATE(),
    NULL,
    1
  FROM stg_listings s
  WHERE
    s.ListingID NOT IN (SELECT ListingID FROM DimListing)
    OR
    s.ListingID IN (
      SELECT ListingID FROM DimListing
      WHERE IsCurrent = 0
      AND CAST(EndDate2 AS DATE) = CAST(GETDATE() AS DATE)
    );
END
```

6.7 sp_Load_DimCampaign

Type: SCD Type 1 **Source:** stg_campaigns **Target:** DimCampaign

```
CREATE PROCEDURE sp_Load_DimCampaign
AS
BEGIN
  -- Update changed records (SCD Type 1)
  UPDATE d
  SET
    d.Channel = ISNULL(s.Channel, 'Unknown'),
    d.StartDate = CAST(s.StartDate AS DATE),
    d.EndDate = CAST(s.EndDate AS DATE),
    d.Cost = ISNULL(s.Cost, 0)
  FROM DimCampaign d
  INNER JOIN stg_campaigns s ON d.CampaignID = s.CampaignID
  WHERE
    d.Channel != ISNULL(s.Channel, 'Unknown') OR
    d.Cost != ISNULL(s.Cost, 0);

  -- Insert new records only
  INSERT INTO DimCampaign
    (CampaignID, ListingID, Channel, StartDate, EndDate, Cost)
  SELECT DISTINCT
    CampaignID,
    ISNULL(ListingID, 'Unknown'),
```

```
    ISNULL(Channel, 'Unknown'),
    CAST(StartDate AS DATE),
    CAST(EndDate AS DATE),
    ISNULL(Cost, 0)
  FROM stg_campaigns
  WHERE CampaignID NOT IN (SELECT CampaignID FROM DimCampaign);
END
```

6.8 sp_Load_FactTransaction

Source: stg_transactions **Target:** FactTransaction

```
CREATE PROCEDURE sp_Load_FactTransaction
AS
BEGIN
  INSERT INTO FactTransaction
    (TransactionID, ListingKey, AgentKey, PropertyKey,
     DateKey, SalePrice, DaysToClose)
  SELECT
    t.TransactionID,
    ISNULL(dl.ListingKey, -1),
    ISNULL(da.AgentKey, -1),
    ISNULL(dp.PropertyKey, -1),
    CAST(FORMAT(CAST(t.ClosingDate AS DATE),
        'yyyyMMdd') AS INT),
    ISNULL(t.SalePrice, 0),
    CASE
      WHEN DATEDIFF(DAY,
        CAST(t.OfferDate AS DATE),
        CAST(t.ClosingDate AS DATE)) < 0
      THEN 0
      ELSE DATEDIFF(DAY,
        CAST(t.OfferDate AS DATE),
        CAST(t.ClosingDate AS DATE))
    END
  FROM (
    SELECT *,
      ROW_NUMBER() OVER (
        PARTITION BY TransactionID
        ORDER BY TransactionID) AS rn
    FROM stg_transactions
  ) t
  LEFT JOIN DimListing dl ON t.ListingID = dl.ListingID
    AND dl.IsCurrent = 1
  LEFT JOIN DimAgent da ON t.AgentID = da.AgentID
```

```
  LEFT JOIN DimProperty dp ON t.PropertyID = dp.PropertyID
  WHERE t.rn = 1
  AND t.TransactionID NOT IN
    (SELECT TransactionID FROM FactTransaction);
END
```

6.9 sp_Load_FactOffer

Source: stg_offers **Target:** FactOffer

```
CREATE PROCEDURE sp_Load_FactOffer
AS
BEGIN
  INSERT INTO FactOffer
    (OfferID, ListingKey, DateKey, OfferPrice, OfferStatus)
  SELECT
    o.OfferID,
    ISNULL(dl.ListingKey, -1),
    CAST(FORMAT(CAST(o.OfferDate AS DATE),
        'yyyyMMdd') AS INT),
    ISNULL(o.OfferPrice, 0),
    ISNULL(o.OfferStatus, 'Unknown')
  FROM (
    SELECT *,
      ROW_NUMBER() OVER (
        PARTITION BY OfferID
        ORDER BY OfferID) AS rn
    FROM stg_offers
  ) o
  LEFT JOIN DimListing dl ON o.ListingID = dl.ListingID
    AND dl.IsCurrent = 1
  WHERE o.rn = 1
  AND o.OfferID NOT IN (SELECT OfferID FROM FactOffer);
END
```

6.10 sp_Load_FactViewing

Source: stg_viewings **Target:** FactViewing

```
CREATE PROCEDURE sp_Load_FactViewing
AS
BEGIN
  INSERT INTO FactViewing
    (ViewingID, ListingKey, DateKey, DurationMinutes, Feedback)
  SELECT
    v.ViewingID,
```

```
    ISNULL(dl.ListingKey, -1),
    CAST(FORMAT(CAST(v.ViewingDate AS DATE),
        'yyyyMMdd') AS INT),
    ISNULL(v.DurationMinutes, 0),
    ISNULL(v.Feedback, 'Unknown')
  FROM (
    SELECT *,
      ROW_NUMBER() OVER (
        PARTITION BY ViewingID
        ORDER BY ViewingID) AS rn
    FROM stg_viewings
  ) v
  LEFT JOIN DimListing dl ON v.ListingID = dl.ListingID
    AND dl.IsCurrent = 1
  WHERE v.rn = 1
  AND v.ViewingID NOT IN (SELECT ViewingID FROM FactViewing);
END
```

6.11 sp_Load_FactLead

Source: stg_leads **Target:** FactLead

```
CREATE PROCEDURE sp_Load_FactLead
AS
BEGIN
  INSERT INTO FactLead
    (LeadID, ListingKey, DateKey, Source, LeadScore)
  SELECT
    l.LeadID,
    ISNULL(dl.ListingKey, -1),
    CAST(FORMAT(CAST(l.LeadDate AS DATE),
        'yyyyMMdd') AS INT),
    ISNULL(l.Source, 'Unknown'),
    ISNULL(l.LeadScore, 0)
  FROM (
    SELECT *,
      ROW_NUMBER() OVER (
        PARTITION BY LeadID
        ORDER BY LeadID) AS rn
    FROM stg_leads
  ) l
  LEFT JOIN DimListing dl ON l.ListingID = dl.ListingID
    AND dl.IsCurrent = 1
  WHERE l.rn = 1
  AND l.LeadID NOT IN (SELECT LeadID FROM FactLead);
```

```
END
```

6.12 sp_InsertPipelineLog

Purpose: Insert pipeline run details into PipelineLog table after each pipeline completes.

```
CREATE PROCEDURE sp_InsertPipelineLog
   @PipelineName   VARCHAR(100),
   @RunStatus      VARCHAR(20),
   @RowsCopied     INT,
   @StartTime      DATETIME,
   @EndTime        DATETIME,
   @ErrorMessage   VARCHAR(500)
AS
BEGIN
   INSERT INTO PipelineLog
      (PipelineName, RunStatus, RowsCopied,
       StartTime, EndTime, ErrorMessage)
   VALUES
      (@PipelineName, @RunStatus, @RowsCopied,
       @StartTime, @EndTime, @ErrorMessage)
END
```

7. Pipelines

7.1 Overview

Three pipelines were created in Azure Data Factory and organized in folders for clean maintainability. All pipelines are version controlled via GitHub repository adf-realestate-analytics.

Pipeline	Folder	Purpose
PL_Copy_RawData_To_Staging	01_Staging	Copy CSV files from Blob to SQL Staging
PL_Load_DWH	02_DWH_Load	Execute SPs to load DWH Dim and Fact tables
PL_Master_RealEstate	03_Master	Orchestrate both pipelines in correct order

7.2 PL_Copy_RawData_To_Staging

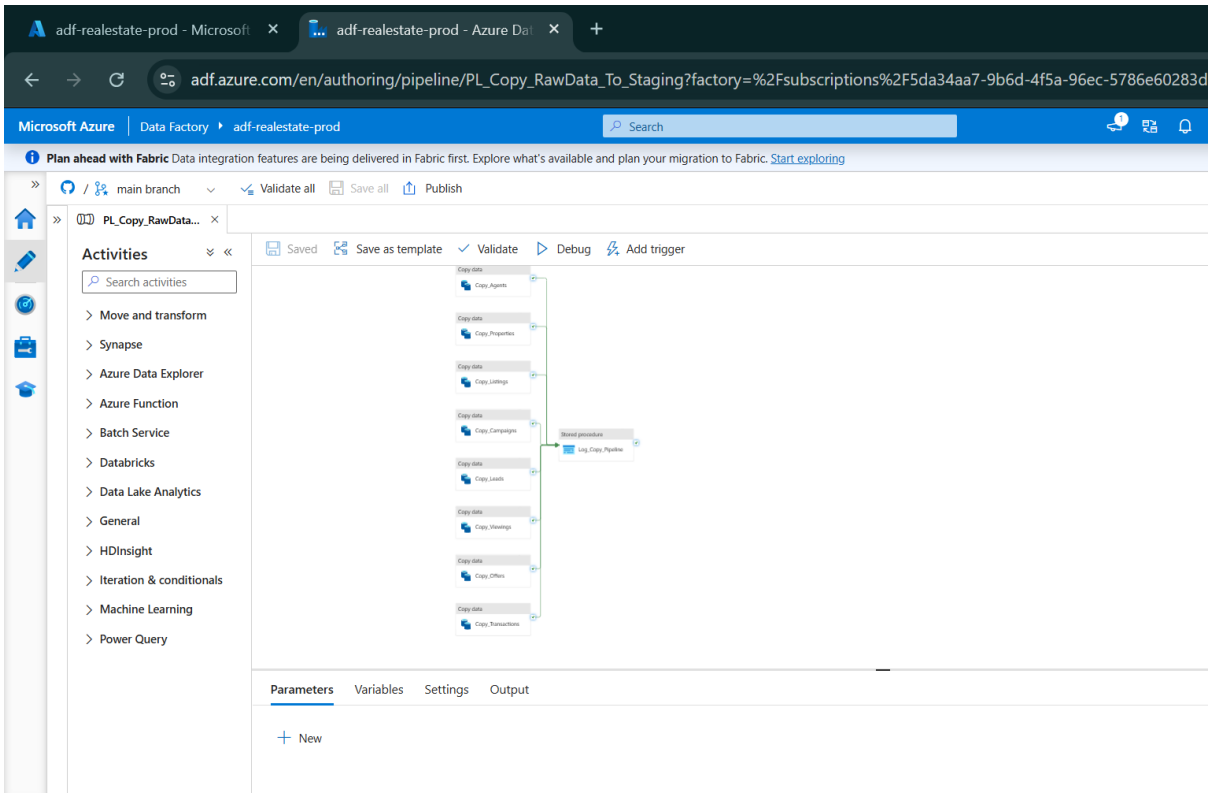
Folder: 01_Staging **Purpose:** Read all 8 CSV files from Blob Storage and load raw data into SQL Staging tables. **Trigger:** Executed by PL_Master_RealEstate via Execute Pipeline activity.

Activities:

Activity Name	Type	Source	Sink	Pre-copy Script
Copy_Agents	Copy Activity	DS_Blob_Agents	DS_SQL_stg_agents	TRUNCATE TABLE stg_agents
Copy_Properties	Copy Activity	DS_Blob_Properties	DS_SQL_stg_properties	TRUNCATE TABLE stg_properties

Copy_Listings	Copy Activity	DS_Blob_Listings	DS_SQL_stg_listings	TRUNCATE TABLE stg_listings
Copy_Campaigns	Copy Activity	DS_Blob_Campaigns	DS_SQL_stg_campaigns	TRUNCATE TABLE stg_campaigns
Copy_Leads	Copy Activity	DS_Blob_Leads	DS_SQL_stg_leads	TRUNCATE TABLE stg_leads
Copy_Viewings	Copy Activity	DS_Blob_Viewings	DS_SQL_stg_viewings	TRUNCATE TABLE stg_viewings
Copy_Offers	Copy Activity	DS_Blob_Offers	DS_SQL_stg_offers	TRUNCATE TABLE stg_offers
Copy_Transactions	Copy Activity	DS_Blob_Transactions	DS_SQL_stg_transactions	TRUNCATE TABLE stg_transactions
Log_Copy_Pipeline	Stored Procedure		sp_InsertPipelineLog	

Execution Flow:



All 8 Copy activities run in parallel simultaneously because all 8 CSV files are completely independent of each other. Parallel execution reduces total pipeline runtime significantly compared to sequential execution. Log activity runs last after all Copy activities complete successfully.

Each staging table is truncated before loading to ensure it contains only the latest data from the current CSV file. This prevents accumulation of old data in staging tables across multiple pipeline runs.

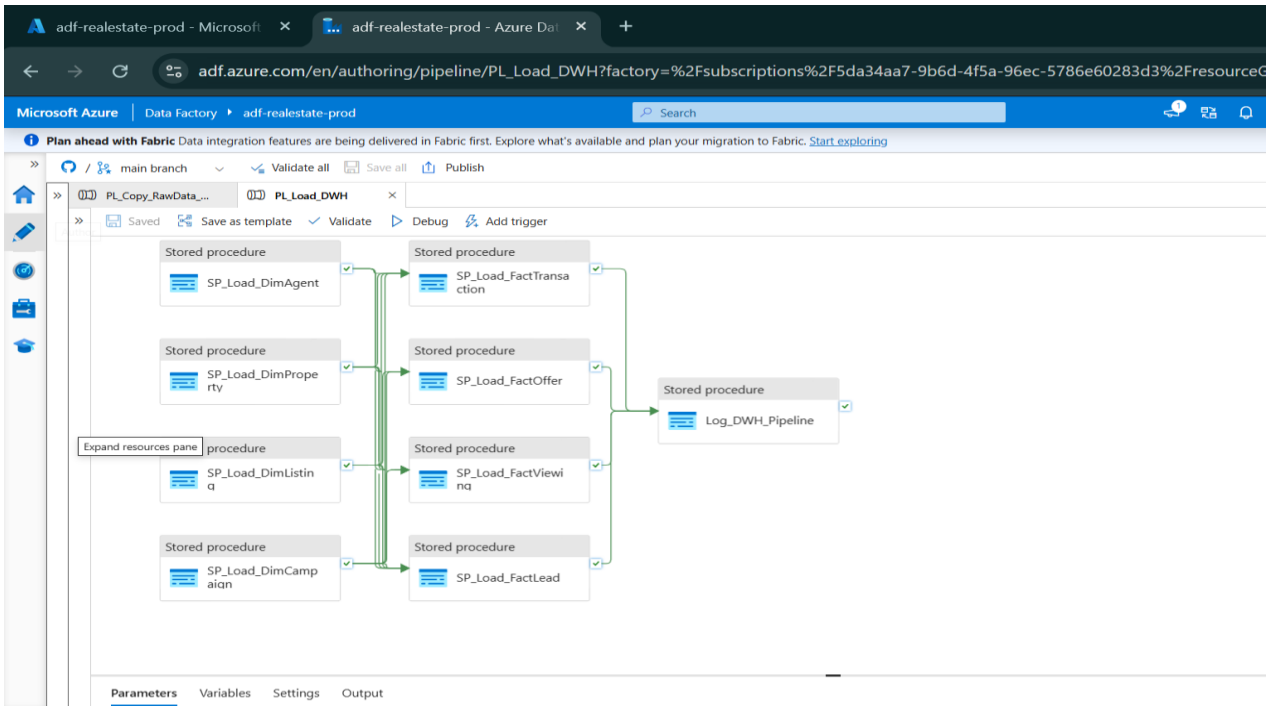
7.3 PL_Load_DWH

Folder: 02_DWH_Load **Purpose:** Execute all 8 Stored Procedures to load clean transformed data from Staging into DWH Dimension and Fact tables. **Trigger:** Executed by PL_Master_RealEstate after PL_Copy_RawData_To_Staging completes.

Activities:

Activity Name	Type	Stored Procedure
SP_Load_DimAgent	Stored Procedure	sp_Load_DimAgent
SP_Load_DimProperty	Stored Procedure	sp_Load_DimProperty
SP_Load_DimListing	Stored Procedure	sp_Load_DimListing
SP_Load_DimCampaign	Stored Procedure	sp_Load_DimCampaign
SP_Load_FactTransaction	Stored Procedure	sp_Load_FactTransaction
SP_Load_FactOffer	Stored Procedure	sp_Load_FactOffer
SP_Load_FactViewing	Stored Procedure	sp_Load_FactViewing
SP_Load_FactLead	Stored Procedure	sp_Load_FactLead
Log_DWH_Pipeline	Stored Procedure	sp_InsertPipelineLog

Execution Flow:



Dimension tables must be fully loaded before Fact tables because Fact tables reference surrogate keys from Dimension tables via foreign key constraints. If Facts ran before Dims, the surrogate key

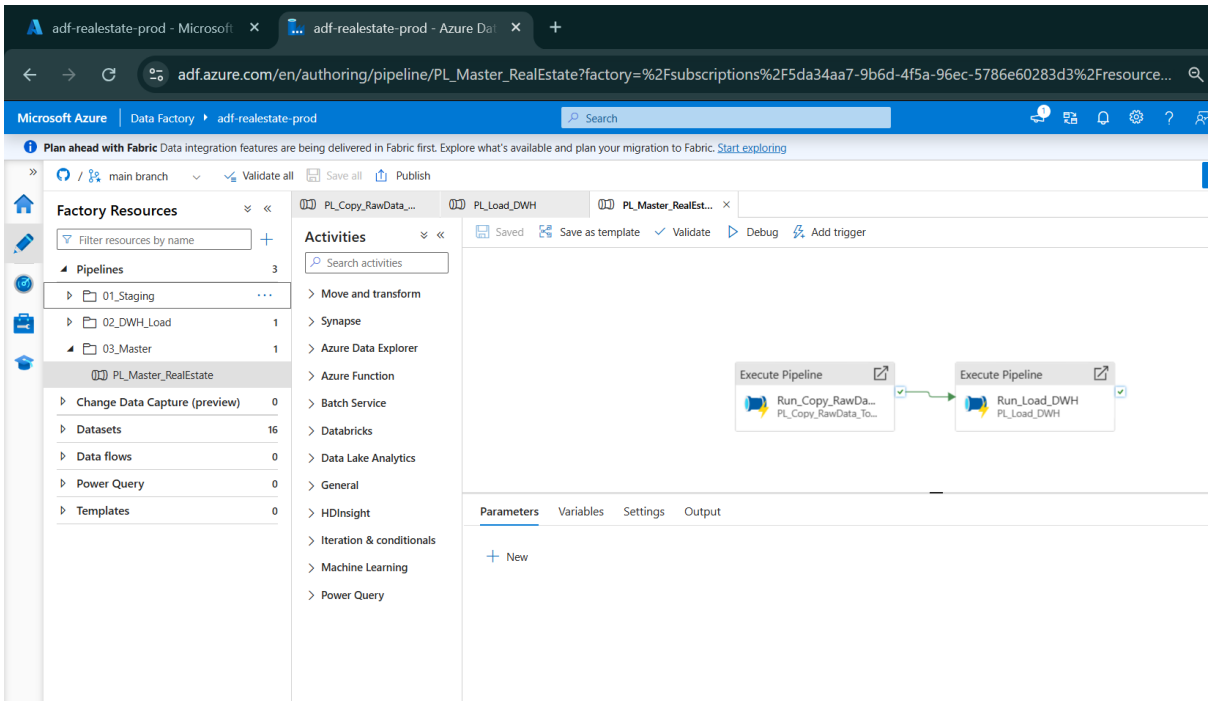
lookup joins would return NULL values resulting in incorrect data. All 4 Dim SPs run in parallel first. Once all complete successfully all 4 Fact SPs run in parallel.

7.4 PL_Master_RealEstate

Folder: 03_Master **Purpose:** Master orchestration pipeline that runs both child pipelines in correct sequence with a single trigger. **Trigger:** Manual trigger or scheduled trigger.

Activity Name	Type	Invoked Pipeline	Wait on Completion
Run_Copy_RawData_To_Staging	Execute Pipeline	PL_Copy_RawData_To_Staging	Yes
Run_Load_DWH	Execute Pipeline	PL_Load_DWH	Yes

Execution Flow:



Wait on Completion ensures PL_Copy_RawData_To_Staging fully completes before PL_Load_DWH starts. This guarantees all staging tables have fresh data before Stored Procedures run. If Wait on Completion was No both pipelines would run simultaneously causing Stored Procedures to read empty or incomplete staging tables.

Instead of triggering two pipelines separately a single trigger on the Master Pipeline runs everything in correct order automatically. This is the standard enterprise pattern for pipeline orchestration in ADF. It also simplifies monitoring — only one pipeline run needs to be checked in ADF Monitor.

7.5 Pipeline Logging

Purpose: Track every pipeline run with status, row count, start time, and end time for monitoring and audit purposes.

How It Works: After each pipeline completes successfully a Stored Procedure activity executes sp_InsertPipelineLog and inserts one row into the PipelineLog table with the following details:

Parameter	Value
PipelineName	Name of pipeline that ran
RunStatus	Success
RowsCopied	Total rows processed
StartTime	@{utcNow()} — ADF system variable
EndTime	@{utcNow()} — ADF system variable
ErrorMessage	None

7.6 GitHub Version Control

All ADF pipeline definitions are stored as JSON files in GitHub repository adf-realestate-analytics. Every time changes are published in ADF Studio they are committed to the main branch automatically.

8. Data Quality

8.1 Overview

Data quality checks were performed at two stages. First during initial data analysis before pipeline development to understand source data issues. Second after pipeline execution to validate loaded data in DWH tables. All issues found were resolved either in Stored Procedures or via direct SQL fixes.

8.2 Validation Queries and Results

Five validation queries were executed after full pipeline run to confirm data quality.

Validation 1 — Row Count Check

```
SELECT 'DimAgent' AS TableName, COUNT(*) AS [RowCount] FROM DimAgent
UNION ALL
SELECT 'DimProperty', COUNT(*) FROM DimProperty
UNION ALL
SELECT 'DimListing', COUNT(*) FROM DimListing
UNION ALL
SELECT 'DimCampaign', COUNT(*) FROM DimCampaign
UNION ALL
SELECT 'DimDate', COUNT(*) FROM DimDate
UNION ALL
SELECT 'FactTransaction', COUNT(*) FROM FactTransaction
UNION ALL
SELECT 'FactOffer', COUNT(*) FROM FactOffer
UNION ALL
SELECT 'FactViewing', COUNT(*) FROM FactViewing
UNION ALL
SELECT 'FactLead', COUNT(*) FROM FactLead
```

Result:

Table	Row Count
DimAgent	500
DimProperty	8,000
DimListing	12,000
DimCampaign	1,500
DimDate	1,461
FactTransaction	8,472
FactOffer	25,000
FactViewing	40,000
FactLead	50,000

Validation 2 — NULL Check on FactTransaction

```
SELECT
    SUM(CASE WHEN ListingKey IS NULL THEN 1 ELSE 0 END) AS Null_ListingKey,
    SUM(CASE WHEN AgentKey IS NULL THEN 1 ELSE 0 END) AS Null_AgentKey,
    SUM(CASE WHEN PropertyKey IS NULL THEN 1 ELSE 0 END) AS Null_PropertyKey,
    SUM(CASE WHEN DateKey IS NULL THEN 1 ELSE 0 END) AS Null_DateKey,
    SUM(CASE WHEN SalePrice IS NULL THEN 1 ELSE 0 END) AS Null_SalePrice
FROM FactTransaction
```

Result:

Column	NULL Count
Null_ListingKey	0
Null_AgentKey	0
Null_PropertyKey	0
Null_DateKey	0
Null_SalePrice	0

Validation 3 — Orphan Key Check

```
SELECT 'FactTransaction orphan ListingKey' AS Check_Name,
    COUNT(*) AS IssueCount
FROM FactTransaction ft
WHERE ft.ListingKey NOT IN (SELECT ListingKey FROM DimListing)
UNION ALL
SELECT 'FactOffer orphan ListingKey',
    COUNT(*)
FROM FactOffer fo
WHERE fo.ListingKey NOT IN (SELECT ListingKey FROM DimListing)
UNION ALL
SELECT 'FactViewing orphan ListingKey',
    COUNT(*)
FROM FactViewing fv
WHERE fv.ListingKey NOT IN (SELECT ListingKey FROM DimListing)
UNION ALL
SELECT 'FactLead orphan ListingKey',
```

```
    COUNT(*)
FROM FactLead fl
WHERE fl.ListingKey NOT IN (SELECT ListingKey FROM DimListing)
```

Result:

Check	Issue Count
FactTransaction orphan ListingKey	0
FactOffer orphan ListingKey	0
FactViewing orphan ListingKey	0
FactLead orphan ListingKey	0

Validation 4 — Date Range Check

```
SELECT
    MIN(dd.FullDate) AS Earliest_Date,
    MAX(dd.FullDate) AS Latest_Date,
    COUNT(DISTINCT dd.DateKey) AS Total_Dates
FROM FactTransaction ft
LEFT JOIN DimDate dd ON ft.DateKey = dd.DateKey
```

Result:

Metric	Value
Earliest Date	2023-01-10
Latest Date	2026-01-11
Total Distinct Dates	1,079

Validation 5 — Business Logic Check

```
SELECT
    da.Region,
    COUNT(ft.TransactionKey) AS TotalSales,
    AVG(ft.DaysToClose) AS AvgDaysToClose,
    SUM(ft.SalePrice) AS TotalRevenue,
    MIN(ft.SalePrice) AS MinSalePrice,
    MAX(ft.SalePrice) AS MaxSalePrice
FROM FactTransaction ft
LEFT JOIN DimAgent da ON ft.AgentKey = da.AgentKey
GROUP BY da.Region
ORDER BY TotalRevenue DESC
```

Result:

Region	Total Sales	Avg Days to Close	Total Revenue
APAC	1,445	46	$1,704,006,816
LATAM	1,561	47	$1,618,146,457
North America	1,300	47	$1,493,592,356
Middle East	1,366	47	$1,491,354,721

Europe	1,394	46	$1,471,098,836
Africa	1,406	47	$1,431,312,087

9. SCD Implementation

9.1 What is SCD

SCD stands for Slowly Changing Dimension. In a data warehouse dimension data does not stay the same forever. Over time attributes like an agent's city, a property's type, or a listing's status can change. SCD defines the strategy for handling these changes in the data warehouse.

Without SCD implementation every pipeline run would either:

- Skip the changed record because it already exists — old incorrect value stays forever
- Insert a duplicate record causing data integrity issues

SCD solves this problem by defining clear rules for how changes are handled.

9.2 SCD Types Implemented

Table	SCD Type	Reason
DimAgent	Type 1 — Overwrite	Agent city or region change — only current location needed for reporting
DimProperty	Type 1 — Overwrite	Property details change — current details are correct values for analytics
DimCampaign	Type 1 — Overwrite	Campaign cost update — latest cost is correct value for ROI calculation
DimListing	Type 2 — History	Listing status changes (Active → Sold) are critical business events — full history must be preserved

9.3 SCD Type 1 — Detail

What It Does: When source data has a changed value for an existing record the old value in DWH is overwritten with the new value. No history of the old value is kept.

Tables: DimAgent, DimProperty, DimCampaign

Columns Monitored for Changes:

DimAgent:

- Region, City, ExperienceYears

DimProperty:

- Region, City, PropertyType, Bedrooms, Bathrooms

DimCampaign:

- Channel, Cost

9.4 SCD Type 2 — Detail

What It Does: When source data has a changed value for an existing record a new row is inserted with the new values. The old row is marked as expired with an end date. Both old and new records coexist in the table. Full history is preserved.

Table: DimListing

Additional Columns Added:

Column	Data Type	Purpose
StartDate	DATE	Date this version of record became active
EndDate2	DATE	Date this version expired — NULL if still current
IsCurrent	BIT	1 = current active record, 0 = historical expired record

Columns Monitored for Changes:

- Status
- AskingPrice
- Currency
- EndDate

9.5 How Analytics Team Uses SCD Type 2

```
SELECT * FROM DimListing
WHERE IsCurrent = 1
```

Get Full History of a Specific Listing:

```
SELECT ListingID, Status, AskingPrice,
    StartDate, EndDate2, IsCurrent
FROM DimListing
WHERE ListingID = 'LS000001'
ORDER BY StartDate
```

Count How Many Times Listings Changed Status:

```
SELECT ListingID, COUNT(*) AS StatusChanges
FROM DimListing
GROUP BY ListingID
HAVING COUNT(*) > 1
ORDER BY StatusChanges DESC
```

10. Analytical Views

10.1 Overview

Four SQL Views were created on top of DWH tables to provide the analytics team with clean, pre-joined, and ready to use datasets. Views eliminate the need for the analytics team to understand the underlying star schema or write complex JOIN queries. They simply connect to the view and start building reports immediately.

Why Views Instead of Flat Tables?

Reason	Explanation
Always Current	Views query live DWH data — no separate refresh needed
No Storage Cost	Views do not store data — zero additional storage
Simple for Analytics Team	Pre-joined columns from multiple tables in one place
Maintainable	If DWH structure changes only view definition needs updating
Security	Analytics team can be granted access to views only — not underlying tables

10.2 Performance Optimization

Seven indexes were created on foreign key columns in all Fact tables to improve view query performance. Without indexes SQL Server performs full table scans on 145,000+ rows for every query. With indexes SQL Server uses index seek which is significantly faster.

```
CREATE INDEX IX_FactTransaction_ListingKey
ON FactTransaction(ListingKey);

CREATE INDEX IX_FactTransaction_AgentKey
ON FactTransaction(AgentKey);

CREATE INDEX IX_FactTransaction_PropertyKey
ON FactTransaction(PropertyKey);

CREATE INDEX IX_FactOffer_ListingKey
ON FactOffer(ListingKey);

CREATE INDEX IX_FactViewing_ListingKey
ON FactViewing(ListingKey);

CREATE INDEX IX_FactLead_ListingKey
ON FactLead(ListingKey);

CREATE INDEX IX_DimListing_ListingID
ON DimListing(ListingID);
```

10.3 vw_AgentPerformance

Purpose: Provides a complete view of each agent's transaction history including sale prices, days to close, listing details, and date dimensions. Enables analytics team to build agent productivity dashboards, regional performance comparisons, and commission calculations.

```
CREATE VIEW vw_AgentPerformance AS
SELECT
    da.AgentKey,
    da.AgentID,
    da.AgentName,
    da.Region,
    da.City,
    da.ExperienceYears,
    ft.TransactionID,
    ft.SalePrice,
    ft.DaysToClose,
    dl.ListingID,
    dl.Status,
    dl.AskingPrice,
    dl.Currency,
    dd.FullDate AS ClosingDate,
    dd.Month,
    dd.MonthName,
    dd.Quarter,
    dd.Year
FROM FactTransaction ft
LEFT JOIN DimAgent da ON ft.AgentKey = da.AgentKey
LEFT JOIN DimListing dl ON ft.ListingKey = dl.ListingKey
LEFT JOIN DimDate dd ON ft.DateKey = dd.DateKey
```

Column Reference:

Column	Source Table	Description
AgentKey	DimAgent	Surrogate key
AgentID	DimAgent	Natural key
AgentName	DimAgent	Full agent name
Region	DimAgent	Geographic region
City	DimAgent	Agent city
ExperienceYears	DimAgent	Years of experience
TransactionID	FactTransaction	Natural transaction key
SalePrice	FactTransaction	Final sale price
DaysToClose	FactTransaction	Days from offer to closing
ListingID	DimListing	Natural listing key
Status	DimListing	Listing status
AskingPrice	DimListing	Original asking price
Currency	DimListing	Price currency

ClosingDate	DimDate	Full closing date
Month	DimDate	Month number
MonthName	DimDate	Month name
Quarter	DimDate	Quarter number
Year	DimDate	Year

10.4 vw_PropertySales

Purpose: Provides a complete view of property details combined with transaction data. Enables analytics team to analyze which property types sell fastest, which regions have highest sale prices, and how property characteristics affect sale price.

```
CREATE VIEW vw_PropertySales AS
SELECT
    dp.PropertyKey,
    dp.PropertyID,
    dp.PropertyType,
    dp.Region,
    dp.City,
    dp.Bedrooms,
    dp.Bathrooms,
    dp.AreaSqft,
    dp.YearBuilt,
    dl.ListingID,
    dl.AskingPrice,
    dl.Currency,
    dl.Status,
    ft.TransactionID,
    ft.SalePrice,
    ft.DaysToClose,
    dd.FullDate AS ClosingDate,
    dd.Month,
    dd.MonthName,
    dd.Quarter,
    dd.Year
FROM FactTransaction ft
LEFT JOIN DimProperty dp ON ft.PropertyKey = dp.PropertyKey
LEFT JOIN DimListing dl ON ft.ListingKey = dl.ListingKey
LEFT JOIN DimDate dd ON ft.DateKey = dd.DateKey
```

Column Reference:

Column	Source Table	Description
PropertyKey	DimProperty	Surrogate key
PropertyID	DimProperty	Natural key
PropertyType	DimProperty	Apartment, Villa, Plot etc

Region	DimProperty	Geographic region
City	DimProperty	Property city
Bedrooms	DimProperty	Number of bedrooms
Bathrooms	DimProperty	Number of bathrooms
AreaSqft	DimProperty	Area in square feet
YearBuilt	DimProperty	Year property was built
ListingID	DimListing	Natural listing key
AskingPrice	DimListing	Original asking price
Currency	DimListing	Price currency
Status	DimListing	Listing status
TransactionID	FactTransaction	Natural transaction key
SalePrice	FactTransaction	Final sale price
DaysToClose	FactTransaction	Days from offer to closing
ClosingDate	DimDate	Full closing date
Month	DimDate	Month number
MonthName	DimDate	Month name
Quarter	DimDate	Quarter number
Year	DimDate	Year

10.5 vw_CampaignEffectiveness

Purpose: Provides a complete view of marketing campaign details combined with lead data. Enables analytics team to measure campaign ROI, compare channel effectiveness, and understand which campaigns generate the highest quality leads.

```
CREATE VIEW vw_CampaignEffectiveness AS
SELECT
    dc.CampaignKey,
    dc.CampaignID,
    dc.Channel,
    dc.StartDate,
    dc.EndDate,
    dc.Cost,
    dl.ListingID,
    dl.Status,
    dl.AskingPrice,
    fl.LeadID,
    fl.Source,
    fl.LeadScore,
    dd.FullDate AS LeadDate,
    dd.Month,
    dd.MonthName,
    dd.Quarter,
    dd.Year
FROM DimCampaign dc
```

```
LEFT JOIN DimListing dl ON dc.ListingID = dl.ListingID
LEFT JOIN FactLead fl ON dl.ListingKey = fl.ListingKey
LEFT JOIN DimDate dd ON fl.DateKey = dd.DateKey
```

Column Reference:

Column	Source Table	Description
CampaignKey	DimCampaign	Surrogate key
CampaignID	DimCampaign	Natural key
Channel	DimCampaign	Marketing channel
StartDate	DimCampaign	Campaign start date
EndDate	DimCampaign	Campaign end date
Cost	DimCampaign	Campaign cost
ListingID	DimListing	Natural listing key
Status	DimListing	Listing status
AskingPrice	DimListing	Original asking price
LeadID	FactLead	Natural lead key
Source	FactLead	Lead source channel
LeadScore	FactLead	Lead quality score
LeadDate	DimDate	Full lead date
Month	DimDate	Month number
MonthName	DimDate	Month name
Quarter	DimDate	Quarter number
Year	DimDate	Year

10.6 vw_SalesFunnel

Purpose: Provides a complete end to end sales funnel view for every listing showing total leads, viewings, offers, and transactions. Enables analytics team to measure conversion rates at every stage of the sales funnel and identify bottlenecks.

```
CREATE VIEW vw_SalesFunnel AS
SELECT
   dl.ListingID,
   dl.Status,
   dl.AskingPrice,
   dl.Currency,
   dl.ListDate,
   da.AgentName,
   da.Region,
   COUNT(DISTINCT fl.LeadKey) AS TotalLeads,
   COUNT(DISTINCT fv.ViewingKey) AS TotalViewings,
   COUNT(DISTINCT fo.OfferKey) AS TotalOffers,
   COUNT(DISTINCT ft.TransactionKey) AS TotalTransactions,
   SUM(ft.SalePrice) AS TotalSalePrice,
   AVG(ft.DaysToClose) AS AvgDaysToClose
```

```
FROM DimListing dl
LEFT JOIN DimAgent da ON dl.AgentID = da.AgentID
LEFT JOIN FactLead fl ON dl.ListingKey = fl.ListingKey
LEFT JOIN FactViewing fv ON dl.ListingKey = fv.ListingKey
LEFT JOIN FactOffer fo ON dl.ListingKey = fo.ListingKey
LEFT JOIN FactTransaction ft ON dl.ListingKey = ft.ListingKey
GROUP BY
    dl.ListingID,
    dl.Status,
    dl.AskingPrice,
    dl.Currency,
    dl.ListDate,
    da.AgentName,
    da.Region
```

Column Reference:

Column	Source Table	Description
ListingID	DimListing	Natural listing key
Status	DimListing	Current listing status
AskingPrice	DimListing	Original asking price
Currency	DimListing	Price currency
ListDate	DimListing	Date listed
AgentName	DimAgent	Agent managing listing
Region	DimAgent	Agent region
TotalLeads	FactLead	Count of leads for listing
TotalViewings	FactViewing	Count of viewings for listing
TotalOffers	FactOffer	Count of offers for listing
TotalTransactions	FactTransaction	Count of transactions for listing
TotalSalePrice	FactTransaction	Sum of sale prices
AvgDaysToClose	FactTransaction	Average days to close

Note: TotalSalePrice and AvgDaysToClose will show NULL for listings with no completed transactions. This is expected and correct behavior — listings with status Active, Expired, or Withdrawn have not completed a sale.

11. Data Dictionary

11.1 Overview

This section provides a complete reference of every table and column in the data warehouse including data type, constraints, description, and example values. This is the primary reference document for the analytics team when building reports and dashboards.

11.2 DimAgent

Column	Data Type	Constraint	Description	Example

AgentKey	INT	PK, IDENTITY(1,1)	System generated surrogate key	1
AgentID	VARCHAR(10)	No	Natural key from CRM source system	AG0001
AgentName	VARCHAR(100)	No	Full name of real estate agent	Agent 0001
Region	VARCHAR(50)	No	Geographic sales region — NULL replaced with Unknown	Europe
City	VARCHAR(50)	No	City where agent is based — NULL replaced with Unknown	Paris
JoinDate	DATE	No	Date agent joined the organization	2023-05-15
ExperienceYears	INT	No	Years of real estate experience — NULL replaced with 0	5

11.3 DimProperty

Column	Data Type	Constraint	Description	Example
PropertyKey	INT	PK, IDENTITY(1,1)	System generated surrogate key	1
PropertyID	VARCHAR(10)	No	Natural key from property management system	PR00001
Region	VARCHAR(50)	No	Geographic region of property	Europe
City	VARCHAR(50)	No	City where property is located	Paris
PropertyType	VARCHAR(50)	No	Type of property — Apartment, Villa, Townhouse, Plot, Office, Retail, Studio	Apartment
Bedrooms	INT	No	Number of bedrooms — NULL replaced with 0 (0 indicates unknown)	2
Bathrooms	INT	No	Number of bathrooms — NULL replaced with 0	1
AreaSqft	INT	No	Total area of property in square feet	898
YearBuilt	INT	No	Year property was constructed	1981

11.4 DimListing

Column	Data Type	Constraint	Description	Example
ListingKey	INT	PK, IDENTITY(1,1)	System generated surrogate key	1
ListingID	VARCHAR(10)	No	Natural key from property management system	LS000001

PropertyID	VARCHAR(10)	No	Natural key reference to property	PR03737
AgentID	VARCHAR(10)	No	Natural key reference to agent	AG0227
ListDate	DATE	No	Date property was listed for sale	2025-08-27
Status	VARCHAR(20)	No	Current status of listing — Active, Sold, Expired, Withdrawn, Under Offer	Active
AskingPrice	DECIMAL(18,2)	No	Listed asking price of property	146357.00
Currency	VARCHAR(5)	No	Currency of asking price — USD, EUR, GBP, AUD, SGD, CAD, JPY	USD
EndDate	DATE	Yes	Date listing ended — NULL for Active listings	2023-07-27
StartDate	DATE	No	SCD2 — Date this version of record became active	2025-08-27
EndDate2	DATE	Yes	SCD2 — Date this version expired — NULL if current record	NULL
IsCurrent	BIT	No	SCD2 — 1 = current active record, 0 = historical expired record	1

11.5 DimCampaign

Column	Data Type	Constraint	Description	Example
CampaignKey	INT	PK, IDENTITY(1,1)	System generated surrogate key	1
CampaignID	VARCHAR(10)	No	Natural key from marketing system	MC00001
ListingID	VARCHAR(10)	No	Natural key reference to listing	LS004238
Channel	VARCHAR(50)	No	Marketing channel used — Portal Featured, Search Ads, Email Blast, Social Ads, Offline	Portal Featured
StartDate	DATE	No	Date campaign started	2025-04-29
EndDate	DATE	No	Date campaign ended	2025-05-27
Cost	DECIMAL(18,2)	No	Total cost of campaign in USD	24693.33

11.6 DimDate

Column	Data Type	Constraint	Description	Example

DateKey	INT	PK	Date in YYYYMMDD format used as foreign key in Fact tables	20230101
FullDate	DATE	No	Full date value	2023-01-01
Day	INT	No	Day number within month (1 to 31)	1
Month	INT	No	Month number (1 to 12)	1
MonthName	VARCHAR(20)	No	Full month name	January
Quarter	INT	No	Quarter number (1 to 4)	1
Year	INT	No	Four digit year	2023

11.7 FactTransaction

Column	Data Type	Constraint	Description	Example
TransactionKey	INT	PK, IDENTITY(1,1)	System generated surrogate key	1
TransactionID	VARCHAR(10)	No	Natural key from sales system	TR0000001
ListingKey	INT	FK → DimListing	Surrogate key reference to DimListing	9562
AgentKey	INT	FK → DimAgent	Surrogate key reference to DimAgent	96
PropertyKey	INT	FK → DimProperty	Surrogate key reference to DimProperty	222
DateKey	INT	FK → DimDate	Closing date in YYYYMMDD format	20240424
SalePrice	DECIMAL(18,2)	No	Final agreed sale price of property	168333.00
DaysToClose	INT	No	Number of days from offer to closing — negative values set to 0	41

11.8 FactOffer

Column	Data Type	Constraint	Description	Example
OfferKey	INT	PK, IDENTITY(1,1)	System generated surrogate key	1
OfferID	VARCHAR(10)	No	Natural key from sales system	OF000001
ListingKey	INT	FK → DimListing	Surrogate key reference to DimListing	11458
DateKey	INT	FK → DimDate	Offer date in YYYYMMDD format	20240722
OfferPrice	DECIMAL(18,2)	No	Price offered by buyer	155298.00
OfferStatus	VARCHAR(20)	No	Status of offer — Accepted, Rejected, Counter	Rejected

11.9 FactViewing

Column	Data Type	Constraint	Description	Example
ViewingKey	INT	PK, IDENTITY(1,1)	System generated surrogate key	1
ViewingID	VARCHAR(10)	No	Natural key from property management system	VW0000001
ListingKey	INT	FK → DimListing	Surrogate key reference to DimListing	10783
DateKey	INT	FK → DimDate	Viewing date in YYYYMMDD format	20230718
DurationMinutes	INT	No	Duration of property viewing in minutes	65
Feedback	VARCHAR(20)	No	Viewer feedback after viewing — Positive, Neutral, Negative	Neutral

11.10 FactLead

Column	Data Type	Constraint	Description	Example
LeadKey	INT	PK, IDENTITY(1,1)	System generated surrogate key	1
LeadID	VARCHAR(10)	No	Natural key from CRM system	LD0000001
ListingKey	INT	FK → DimListing	Surrogate key reference to DimListing	10456
DateKey	INT	FK → DimDate	Lead date in YYYYMMDD format	20230405
Source	VARCHAR(50)	No	Channel through which lead was generated — Portal, Website, Email, Referral, Walk-in, Social, Phone	Portal
LeadScore	DECIMAL(5,2)	No	Quality score of lead from 1 to 100 — higher score means higher quality	3.00

11.11 PipelineLog

Column	Data Type	Constraint	Description	Example
LogID	INT	PK, IDENTITY(1,1)	Auto generated log record ID	1
PipelineName	VARCHAR(100)	No	Name of ADF pipeline that ran	PL_Copy_RawData_To_Staging
RunStatus	VARCHAR(20)	No	Execution status of pipeline	Success

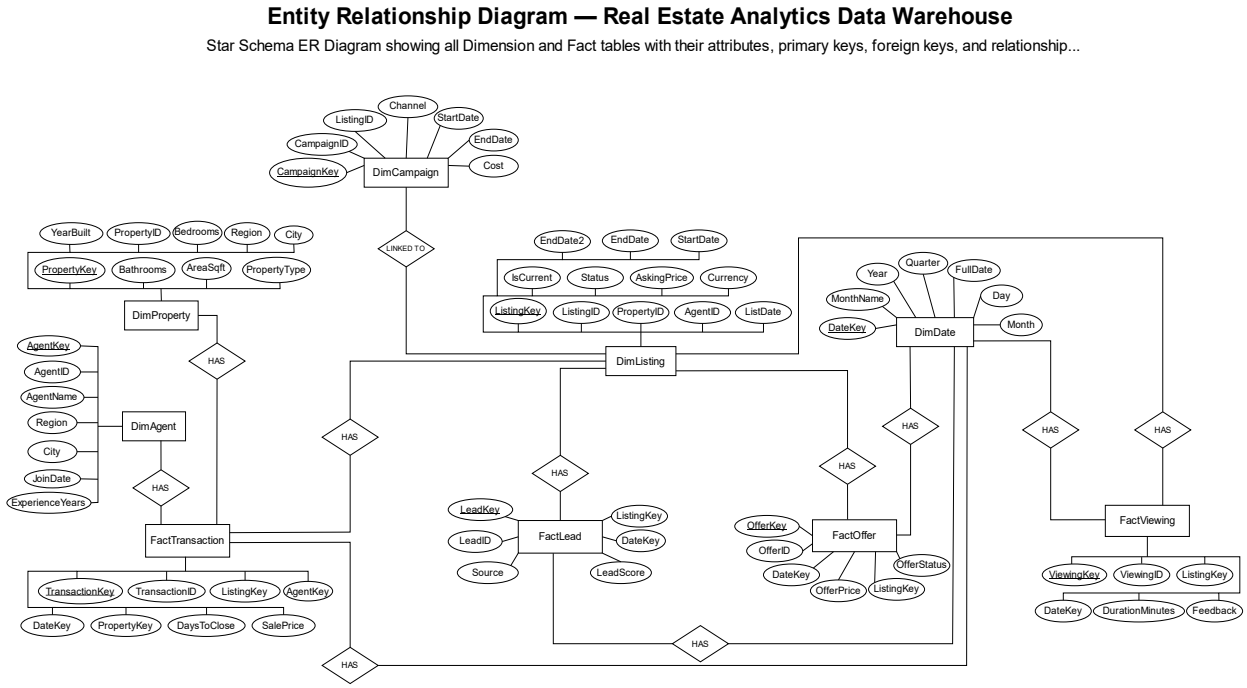
RowsCopied	INT	No	Total number of rows processed in this run	145532
StartTime	DATETIME	No	Pipeline execution start time in UTC	2026-05-20 10:00:00
EndTime	DATETIME	No	Pipeline execution end time in UTC	2026-05-20 10:05:00
ErrorMessage	VARCHAR(500)	No	Error details if pipeline failed — NULL if successful	None

11.12 Staging Tables Reference

All staging tables mirror source CSV structure exactly. No transformations applied. All date columns stored as VARCHAR to accept any format from source system.

Staging Table	Source File	Columns	Row Count
stg_agents	Agents.csv	AgentID, AgentName, Region, City, JoinDate, ExperienceYears	500
stg_properties	Properties.csv	PropertyID, Region, City, PropertyType, Bedrooms, Bathrooms, AreaSqft, YearBuilt	8,000
stg_listings	Listings.csv	ListingID, PropertyID, AgentID, ListDate, Status, AskingPrice, Currency, EndDate	12,060
stg_campaigns	Campaigns.csv	CampaignID, ListingID, StartDate, EndDate, Channel, Cost	1,500
stg_leads	Leads.csv	LeadID, ListingID, LeadDate, Source, LeadScore	50,000
stg_viewings	Viewings.csv	ViewingID, ListingID, ViewingDate, DurationMinutes, Feedback	40,000
stg_offers	Offers.csv	OfferID, ListingID, OfferDate, OfferPrice, OfferStatus	25,000
stg_transactions	Transactions.csv	TransactionID, ListingID, AgentID, PropertyID, OfferDate, ClosingDate, SalePrice	8,472

12 ER Diagram



13. Troubleshooting Guide

13.1 Overview

This section covers the most common errors that may occur during pipeline execution, data loading, or SQL operations along with their root causes and step by step fixes.

13.2 SQL Errors

Error: Cannot Truncate Table Referenced by Foreign Key

Field	Detail
Error	Cannot truncate table because it is being referenced by a FOREIGN KEY constraint
Possible Cause	Trying to truncate Dim table while Fact table has data

```
-- Step 1: Truncate Facts first
TRUNCATE TABLE FactTransaction;
TRUNCATE TABLE FactOffer;
TRUNCATE TABLE FactViewing;
TRUNCATE TABLE FactLead;

-- Step 2: Then truncate Dims
TRUNCATE TABLE DimCampaign;
TRUNCATE TABLE DimListing;
TRUNCATE TABLE DimProperty;
TRUNCATE TABLE DimAgent;
```

Error: Duplicate Rows in Fact Tables

Field	Detail
Error	Fact table has more rows than expected
Possible Cause	Pipeline was run multiple times and NOT IN check failed

```
-- Step 1: Truncate affected Fact table
TRUNCATE TABLE FactTransaction; -- or whichever Fact has duplicates

-- Step 2: Re-run sp manually
EXEC sp_Load_FactTransaction;

-- Step 3: Verify count
SELECT COUNT(*) FROM FactTransaction;
```

Error: DELETE Statement Conflicted with REFERENCE Constraint

Field	Detail
Error	DELETE statement conflicted with REFERENCE constraint
Possible Cause	Trying to delete from Dim table that Fact table references

Never delete from Dim tables directly. Instead expire the record using SCD Type 2 approach:

```
-- For DimListing — expire record instead of delete
UPDATE DimListing
SET IsCurrent = 0,
    EndDate2 = GETDATE()
WHERE ListingID = 'LS000001'
AND IsCurrent = 1
```

13.3 Data Quality Issues

Issue 1 — Unexpected NULL Values in DWH

Fix:

```
-- Find NULLs in any Dim table
SELECT * FROM DimAgent WHERE City IS NULL
SELECT * FROM DimProperty WHERE Bedrooms IS NULL
-- Fix NULLs directly
UPDATE DimAgent SET City = 'Unknown' WHERE City IS NULL
UPDATE DimProperty SET Bedrooms = 0 WHERE Bedrooms IS NULL
```

Issue 2 — Row Count Mismatch After Pipeline Run

Fix:

```
-- Compare staging vs DWH counts
SELECT
    'stg_agents' AS Source, COUNT(*) AS Cnt FROM stg_agents
```

```
UNION ALL
SELECT 'DimAgent', COUNT(*) FROM DimAgent
-- If counts differ check for duplicates in staging
SELECT AgentID, COUNT(*)
FROM stg_agents
GROUP BY AgentID
HAVING COUNT(*) > 1
```

13.4 SCD Related Issues

Issue 3 — SCD Type 2 Created Extra Rows Unexpectedly

Fix:

```
-- Check how many versions exist for a listing
SELECT ListingID, COUNT(*) AS Versions
FROM DimListing
GROUP BY ListingID
HAVING COUNT(*) > 1
ORDER BY Versions DESC

-- Check which records are current
SELECT * FROM DimListing
WHERE ListingID = 'LS000001'
ORDER BY StartDate

-- If incorrect version is current fix manually
UPDATE DimListing
SET IsCurrent = 0, EndDate2 = GETDATE()
WHERE ListingID = 'LS000001'
AND IsCurrent = 1
AND Status = 'Sold' -- whichever status is incorrect

UPDATE DimListing
SET IsCurrent = 1, EndDate2 = NULL
WHERE ListingID = 'LS000001'
AND Status = 'Active' -- correct current status
```

13.5 Quick Reference — Common Commands

```
-- Check all table row counts
SELECT 'DimAgent' AS Tbl, COUNT(*) AS Cnt FROM DimAgent
UNION ALL SELECT 'DimProperty', COUNT(*) FROM DimProperty
UNION ALL SELECT 'DimListing', COUNT(*) FROM DimListing
UNION ALL SELECT 'DimCampaign', COUNT(*) FROM DimCampaign
UNION ALL SELECT 'FactTransaction', COUNT(*) FROM FactTransaction
```

```
UNION ALL SELECT 'FactOffer', COUNT(*) FROM FactOffer
UNION ALL SELECT 'FactViewing', COUNT(*) FROM FactViewing
UNION ALL SELECT 'FactLead', COUNT(*) FROM FactLead

-- Check latest pipeline runs
SELECT TOP 5 * FROM PipelineLog ORDER BY LogID DESC

-- Check current listings only
SELECT COUNT(*) FROM DimListing WHERE IsCurrent = 1

-- Check historical listings
SELECT COUNT(*) FROM DimListing WHERE IsCurrent = 0

-- Manually run all SPs in correct order
EXEC sp_Load_DimAgent;
EXEC sp_Load_DimProperty;
EXEC sp_Load_DimListing;
EXEC sp_Load_DimCampaign;
EXEC sp_Load_FactTransaction;
EXEC sp_Load_FactOffer;
EXEC sp_Load_FactViewing;
EXEC sp_Load_FactLead;
```

14. References

ADF GitHub Repository : https://github.com/Vipul2704/adf-realestate-analytics

ER Diagram Tool : https://draw.io